

간단 게시판

□ 개발 흐름

○ 프로젝트 초기화

- `npm init`
- `npm install express ejs express-session mysql2 express-ejs-layouts`

○ 회원 기능 구현

- 회원가입 (이름, 아이디, 비밀번호 저장)
- 로그인 / 로그아웃
- 로그인 상태를 `session`으로 유지

○ 게시판 기능 구현

- 게시글 작성 (제목, 내용, 작성자)
- 게시글 목록 보기
- 게시글 상세 보기
- 게시글 수정 / 삭제 (작성자만 가능)

○ 뷰 구성

- EJS 사용, 전체 스타일은 `style.css` 적용
- 로그인 여부에 따라 메뉴 다르게 표시

□ 프로젝트 구조

simple-board/

```
├── app.js          # 메인 서버 파일
├── db.js           # DB 연결 및 테이블 초기화
├── routes/
│   ├── auth.js    # 회원가입 / 로그인 / 로그아웃
│   └── board.js   # 게시판 CRUD
├── views/
│   ├── layout.ejs # 공통 레이아웃
│   ├── signup.ejs # 회원가입 페이지
│   ├── signup-result.ejs # 회원가입 결과 페이지
│   ├── signin.ejs # 로그인 페이지
│   ├── signin-result.ejs # 로그인 결과 페이지
│   ├── signout-result.ejs # 로그아웃 결과 페이지
│   ├── board-list.ejs # 게시글 목록
│   ├── board-detail.ejs # 게시글 상세
│   ├── board-write.ejs # 글쓰기 폼
│   └── board-edit.ejs # 글수정 폼
├── public/
│   └── style.css   # 공통 CSS
└── package.json   # 패키지 정보
```

□ 주요 화면 흐름

기능	URL 경로	비고
회원가입 폼	GET /signup	signup.ejs
회원가입 처리	POST /signup	DB 저장 / signup-result.ejs
로그인 폼	GET /signin	signin.ejs
로그인 처리	POST /signin	세션 생성 / signin-result.ejs
로그아웃 처리	GET /signout	세션 제거 / signout-result.ejs
게시글 목록	GET /board	list.ejs / 글 목록
글 작성 폼	GET /board/write	write.ejs / 로그인 필요
글 작성 처리	POST /board/write	로그인 필요
글 상세 보기	GET /board/:id	detail.ejs / 조회수 증가 포함 가능
글 수정 폼	GET /board/:id/edit	edit.ejs / 작성자 본인만
글 수정 처리	POST /board/:id/edit	-
글 삭제	GET /board/:id/delete	작성자 본인만

□ **db.js**

```
import mysql from 'mysql2/promise';
```

```
export const pool = mysql.createPool({  
  host: 'localhost',  
  user: 'root',  
  password: '1234',  
  database: 'simple_board',  
  waitForConnections: true,  
  connectionLimit: 5  
});
```

```
export const initTables = async () => {  
  // members와 articles 테이블 생성을 하나의 트랜잭션으로 묶기 위해 getConnection() 사용  
  const conn = await pool.getConnection();  
  try {  
    await conn.query(`  
      CREATE TABLE IF NOT EXISTS members (  
        id INT AUTO_INCREMENT PRIMARY KEY,  
        username VARCHAR(50) UNIQUE NOT NULL,  
        password VARCHAR(100) NOT NULL,  
        created_at DATETIME DEFAULT CURRENT_TIMESTAMP  
      )  
    `);  
  
    await conn.query(`  
      CREATE TABLE IF NOT EXISTS articles (  
        id INT AUTO_INCREMENT PRIMARY KEY,  
        title VARCHAR(100) NOT NULL,  
        content TEXT NOT NULL,  
        writer VARCHAR(50) NOT NULL,  
        created_at DATETIME DEFAULT CURRENT_TIMESTAMP  
      )  
    `);  
  
    console.log("✅ 테이블 생성 완료");  
  } catch (e) {  
    console.error("❌ 테이블 생성 실패:", e.message);  
  } finally {  
    conn.release();  
  }  
};
```

□ routes/auth.js

// 로그인 폼

```
router.get('/signin', (req, res) => {  
  res.render('signin');  
});
```

// 로그인 처리

```
router.post('/signin', async (req, res) => {  
  const { username, password } = req.body;
```

```
  try {  
    const sql = 'SELECT * FROM members WHERE username = ? AND password = ?';  
    const [rows] = await pool.query(sql, [username, password]);  
  
    if (rows.length === 1) {  
      req.session.user = {  
        id: rows[0].id,  
        username: rows[0].username  
      };  
      res.render('signin-result', { success: true, username });  
    } else {  
      res.render('signin-result', { success: false, message: '아이디 또는 비밀번호가 올바르지 않습니다.'  
    });  
    }  
  } catch (err) {  
    console.error('로그인 오류:', err.message);  
    res.render('signin-result', { success: false, message: '서버 오류가 발생했습니다.' });  
  }  
});
```

회원가입

□ app.js

```
import express from 'express';
import session from 'express-session';
import path from 'node:path';
import expressLayouts from 'express-ejs-layouts';
import { initTables } from './db.js';
import authRouter from './routes/auth.js';

const app = express();
const __dirname = import.meta.dirname;

app.use(express.urlencoded());
app.use(express.json());
app.use(express.static(path.join(__dirname, 'public')));

app.use(session({
  secret: 'secret-key',
  resave: false,
  saveUninitialized: false
}));

app.use(expressLayouts);
app.set('layout', 'layout');
app.set('view engine', 'ejs');
app.set('views', path.join(__dirname, 'views'));

await initTables();

app.use('/', authRouter);

app.listen(3000, () => {
  console.log('서버 실행 중: http://localhost:3000');
});
```

□ views/layout.ejs

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8" />
  <title>Simple Board</title>
  <link rel="stylesheet" href="/style.css" />
</head>
<body>
  <div class="container">
    <%- body %>
  </div>
</body>
</html>
```

□ views/signup.ejs

```
<h2>회원가입</h2>
<form method="POST" action="/signup">
  <label>아이디</label>
  <input type="text" name="username" required />
  <label>비밀번호</label>
  <input type="password" name="password" required />
  <button type="submit">가입하기</button>
</form>
```

□ views/signup-result.ejs

```
<% if (success) { %>
  <h2>🎉 <%= username %>님, 회원가입이 완료되었습니다!</h2>
  <p class="message"><a href="/signin">로그인 하러 가기</a></p>
<% } else { %>
  <h2>⚠️ 회원가입에 실패했습니다</h2>
  <p class="message">오류: <%= error %></p>
  <p class="message"><a href="/signup">다시 시도하기</a></p>
<% } %>
```

회원가입

아이디

비밀번호

가입하기

🎉 user님, 회원가입이 완료되었습니다!

[로그인 하러 가기](#)

□ public/style.css

```
body {  
  font-family: Arial, sans-serif; background: #f2f2f2;  
  margin: 0; padding: 0;  
}  
  
.container {  
  width: 400px; margin: 50px auto; background: white;  
  padding: 20px; border-radius: 8px;  
  box-shadow: 2px 2px 10px rgba(0,0,0,0.1);  
}  
  
form {  
  display: flex; flex-direction: column;  
}  
  
label {  
  margin-top: 10px; font-weight: bold;  
}  
  
input[type="text"], input[type="password"],  
textarea {  
  padding: 8px; margin-top: 5px;  
  border: 1px solid #ccc; border-radius: 4px;  
}  
  
textarea {  
  resize: vertical;  
}  
  
button {  
  margin-top: 20px; padding: 10px;  
  background-color: #0066cc;  
  color: white; border: none;  
  border-radius: 4px; cursor: pointer;  
}  
  
button:hover { background-color: #0055aa; }  
  
a { color: #0066cc; text-decoration: none; }  
  
a:hover { text-decoration: underline; }  
  
.message { margin-top: 15px; color: #555; }
```

로그인

□ auth.js

```
import express from 'express';
import session from 'express-session';
import path from 'node:path';
import expressLayouts from 'express-ejs-layouts';
import { initTables } from './db.js';
import authRouter from './routes/auth.js';

const app = express();
const __dirname = import.meta.dirname;

app.use(express.urlencoded());
app.use(express.json());
app.use(express.static(path.join(__dirname, 'public')));

app.use(session({
  secret: 'secret-key',
  resave: false,
  saveUninitialized: false
}));

app.use(expressLayouts);
app.set('layout', 'layout');
app.set('view engine', 'ejs');
app.set('views', path.join(__dirname, 'views'));

await initTables();

app.use('/', authRouter);

app.listen(3000, () => {
  console.log('서버 실행 중: http://localhost:3000');
});
```


□ views/signin.ejs

<h2>로그인</h2>

<form method="POST" action="/signin">

<label>아이디</label>

<input type="text" name="username" required />

<label>비밀번호</label>

<input type="password" name="password" required />

<button type="submit">로그인</button>

</form>

<p style="margin-top: 10px;">

아직 회원이 아니라면? 👉 회원가입

</p>

A login form with a title "로그인" (Login). It contains two input fields: "아이디" (ID) and "비밀번호" (Password). Below the password field is a blue button labeled "로그인" (Login). At the bottom, there is a link "아직 회원이 아니라면? 👉 회원가입" (Not a member yet? 👉 Sign up).

□ views/signin-result.ejs

<% if (success) { %>

<h2>환영합니다, <%= username %>님!</h2>

<p class="message">게시판으로 이동</p>

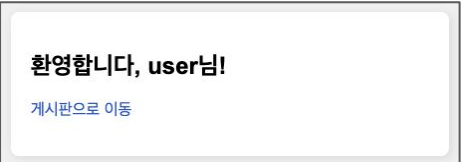
<% } else { %>

<h2>⚠ 로그인 실패</h2>

<p class="message"><%= message %></p>

<p class="message">다시 시도하기</p>

<% } %>

A login result message box. It contains the text "환영합니다, user님!" (Welcome, user!). Below it is a link "게시판으로 이동" (Move to board).

로그아웃

□ 로그아웃 핵심 로직

- 세션에 저장된 사용자 정보 제거 (`req.session.destroy()`)
- 로그아웃 성공 메시지를 EJS로 보여줌

□ auth.js

// 로그아웃 처리

```
router.get('/signout', (req, res) => {  
  req.session.destroy(err => {  
    if (err) {  
      console.error('로그아웃 오류:', err.message);  
      return res.render('signout-result', { success: false });  
    }  
    res.render('signout-result', { success: true });  
  });  
});
```

□ views/signout-result.ejs

```
<% if (success) { %>  
<h2>👋 로그아웃 되었습니다</h2>  
<p class="message"><a href="/signin">다시 로그인하기</a></p>  
<% } else { %>  
<h2>⚠️ 로그아웃 실패</h2>  
<p class="message">잠시 후 다시 시도해 주세요.</p>  
<% } %>
```

👋 로그아웃 되었습니다

[다시 로그인하기](#)

□ app.js

// 현재 로그인 사용자 정보를 템플릿에서 사용할 수 있도록

```
app.use((req, res, next) => {  
  res.locals.user = req.session.user || null;  
  next();  
});
```

(로그아웃을 위한) 사용자 정보 보여주기

□ views/layout.ejs

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8" />
  <title>Simple Board</title>
  <link rel="stylesheet" href="/style.css" />
</head>
<body>
  <div class="container">
    <div class="top-bar">
      <% if (user) { %>
        <span><%= user.username %>님, 안녕하세요! 😊</span>
        <a href="/signout">로그아웃</a>
      <% } %>
    </div>
    <%= body %>
  </div>
</body>
</html>
```



□ public/style.css

```
.top-bar {
  margin-bottom: 20px;
  padding-bottom: 10px;
  border-bottom: 1px solid #ccc;
  font-size: 14px;
  text-align: right;
}
```

□ 새로운 고침을 해야 보여지는 버그 수정 → 화면 새로운 고침 / Redirect

게시글 목록

□ 기능 개요

- 경로: **GET /board**
- 기능: DB에서 모든 게시글을 조회하여 목록으로 보여줌
- 출력 항목: 글 번호, 제목, 작성자, 작성일

□ views/board-list.ejs

<h2>  게시판 </h2>

<% if (articles.length === 0) { %>

<p class="message">작성된 글이 없습니다.</p>

<% } else { %>

<table border="1" cellspacing="0" cellpadding="8" width="100%">

<tr>

<th>번호</th>

<th>제목</th>

<th>작성자</th>

<th>작성일</th>

</tr>

<% articles.forEach(article => { %>

<tr>

<td><%= article.id %></td>

<td><a href="/board/<%= article.id %>"><%= article.title %></td>

<td><%= article.writer %></td>

<td>

<%=

new Date(

new Date(article.created_at).getTime() + 9 * 60 * 60 * 1000

).toLocaleString('ko-KR') %>

</td>

</tr>

<% }) %>

</table>

<% } %>



□ routes/board.js

```
import express from 'express';
import { pool } from '../db.js';

const router = express.Router();

// 게시글 목록 보기
router.get('/', async (req, res) => {
  try {
    const sql = 'SELECT id, title, writer, created_at FROM articles ORDER BY id DESC';
    const [articles] = await pool.query(sql);

    res.render('board-list', { articles });
  } catch (err) {
    console.error('게시글 목록 조회 오류:', err.message);
    res.send('<h2>게시글을 불러오는 데 실패했습니다.</h2>');
  }
});

export default router;
```

□ app.js

```
import authRouter from './routes/auth.js';
import boardRouter from './routes/board.js';

const app = express();
const __dirname = import.meta.dirname;

...

await initTables();

app.use('/', authRouter);
app.use('/board', boardRouter);

app.listen(3000, () => {
  console.log('서버 실행 중: http://localhost:3000');
});
```

게시글 작성

□ 기능 개요

- 경로
 - GET /board/write → 작성 폼
 - POST /board/write → DB 저장 처리
- 로그인한 사용자만 글을 쓸 수 있음 (req.session.user 확인)
- 작성 후 → 게시판 목록(/board)으로 이동

□ views/board-write.ejs

```
<h2><img alt="pencil icon" data-bbox="95 300 115 315"/> 글쓰기 </h2>
<form method="POST" action="/board/write">
  <label>제목</label>
  <input type="text" name="title" required />
  <label>내용</label>
  <textarea name="content" rows="8" required></textarea>
  <button type="submit">작성하기</button>
</form>
<p class="message"><a href="/board">← 목록으로 돌아가기 </a></p>
```

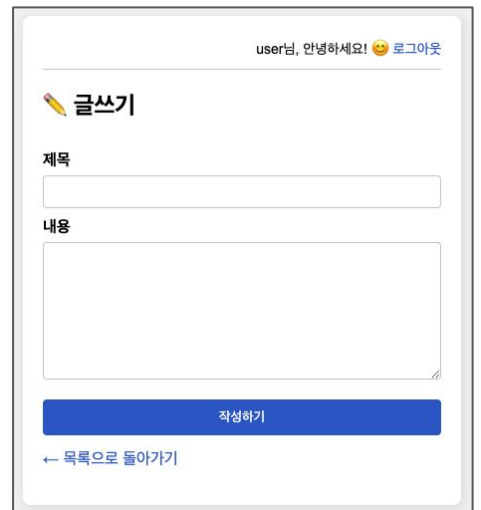
□ routes/board.js

```
router.get('/write', (req, res) => { // 글쓰기 폼
  if (!req.session.user) {
    return res.send('<h2>로그인 후 작성할 수 있습니다. <a href="/signin">로그인 </a></h2>');
  }
  res.render('board-write');
});
```

```
router.post('/write', async (req, res) => { // 글쓰기 처리
  if (!req.session.user) {
    return res.send('<h2>로그인이 필요합니다.</h2>');
  }
}
```

```
const { title, content } = req.body;
const writer = req.session.user.username;
```

```
try {
  const sql = 'INSERT INTO articles (title, content, writer) VALUES (?, ?, ?)';
  await pool.query(sql, [title, content, writer]);
  res.redirect('/board');
} catch (err) {
  console.error('글쓰기 오류:', err.message);
  res.send('<h2>글쓰기 실패! 다시 시도해 주세요.</h2>');
}
});
```



□ views/board-list.ejs

<h2> 📄 게시판 </h2>

<% if (user) { %>

<p style="text-align: right;">+ 새 글 쓰기</p>

<% } %>

<% if (articles.length === 0) { %>

<p class="message">작성된 글이 없습니다.</p>

<% } else { %>

<table border="1" cellspacing="0" cellpadding="8" width="100%">

<tr>

<th>번호</th>

<th>제목</th>

<th>작성자</th>

<th>작성일</th>

</tr>

<% articles.forEach(article => { %>

<tr>

<td><%= article.id %></td>

<td><a href="/board/<%= article.id %>"><%= article.title %></td>

<td><%= article.writer %></td>

<td>

<%=

new Date(

new Date(article.created_at).getTime() + 9 * 60 * 60 * 1000

).toLocaleString('ko-KR')

%>

</td>

</tr>

<% }) %>

</table>

<% } %>



user님, 안녕하세요! 😊 로그아웃			
📄 게시판			
+ 새 글 쓰기			
번호	제목	작성자	작성일
1	test1	user	2025. 5. 1. 오전 9:26:50

게시글 상세보기

□ 기능 개요

- 경로: `GET /board/:id`
- 게시글 번호(id)를 기반으로 해당 글을 DB에서 조회해 화면에 출력

□ views/board-detail.ejs

```
<h2><%= article.title %></h2>
```

```
<p style="color: gray;">
```

```
  작성자: <strong><%= article.writer %></strong> |
```

```
  작성일: <%= new Date(new Date(article.created_at).getTime() + 9 * 60 * 60 * 1000)
            .toLocaleString('ko-KR') %>
```

```
</p>
```

```
<div style="margin-top: 20px; white-space: pre-wrap;">
```

```
  <%= article.content %>
```

```
</div>
```

```
<p class="message"><a href="/board">← 목록으로 돌아가기</a></p>
```

□ routes/board.js

```
// 게시글 상세 보기
```

```
router.get('/:id', async (req, res) => {
```

```
  const { id } = req.params;
```

```
  try {
```

```
    const sql = 'SELECT * FROM articles WHERE id = ?';
```

```
    const [rows] = await pool.query(sql, [id]);
```

```
    if (rows.length === 0) {
```

```
      return res.send('<h2>존재하지 않는 게시글입니다. <a href="/board">목록으로</a></h2>');
```

```
    }
```

```
    const article = rows[0];
```

```
    res.render('board-detail', { article });
```

```
  } catch (err) {
```

```
    console.error('상세보기 오류:', err.message);
```

```
    res.send('<h2>게시글을 불러오는 데 실패했습니다.</h2>');
```

```
  }
```

```
});
```

test1

작성자: user | 작성일: 2025. 5. 1. 오전 9:26:50

test11

[← 목록으로 돌아가기](#)

게시글 수정

□ 기능 개요

- 경로
 - GET /board/:id/edit → 수정 폼
 - POST /board/:id/edit → 수정 처리
- 로그인한 사용자만 가능
- 작성자만 수정 가능

□ views/board-edit.ejs

<h2> 글 수정 </h2>

<form method="POST" action="/board/<%= article.id %>/edit">

<label>제목</label>

<input type="text" name="title" value="<%= article.title %>" required />

<label>내용</label>

<textarea name="content" rows="8" required><%= article.content %></textarea>

<button type="submit">수정 완료</button>

</form>

<p class="message">

<a href="/board/<%= article.id %>">← 돌아가기

</p>

□ views/board-detail.ejs

<div style="margin-top: 20px; white-space: pre-wrap;">

<%= article.content %>

</div>

<% if (user && user.username === article.writer) { %>

<p style="text-align: right;">

<a href="/board/<%= article.id %>/edit"> 수정

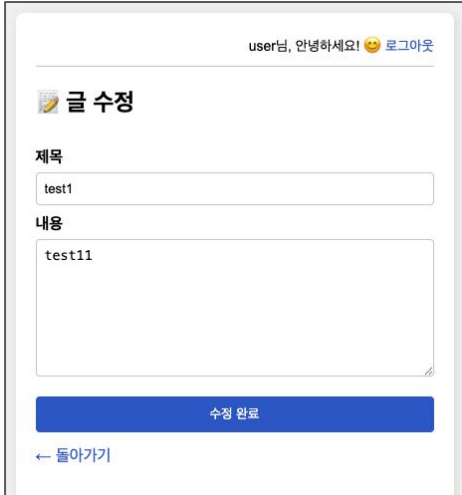
</p>

<% } %>

<p class="message">

← 목록으로 돌아가기

</p>



□ routes/board.js

// 게시글 수정 폼

```
router.get('/:id/edit', async (req, res) => {  
  if (!req.session.user) {  
    return res.send('<h2>로그인 후 수정할 수 있습니다. <a href="/signin">로그인</a></h2>');  
  }  
}
```

```
const { id } = req.params;
```

```
try {  
  const [rows] = await pool.query('SELECT * FROM articles WHERE id = ?', [id]);
```

```
  if (rows.length === 0) {  
    return res.send('<h2>존재하지 않는 게시글입니다.</h2>');  
  }
```

```
  const article = rows[0];
```

```
  if (article.writer !== req.session.user.username) { // 작성자만 수정 가능  
    return res.send('<h2>본인이 작성한 글만 수정할 수 있습니다.</h2>');  
  }
```

```
  res.render('board-edit', { article });  
} catch (err) {  
  console.error('수정 폼 오류:', err.message);  
  res.send('<h2>수정 폼을 불러오는 데 실패했습니다.</h2>');  
}  
});
```

// 게시글 수정 처리

```
router.post('/:id/edit', async (req, res) => {  
  if (!req.session.user) {  
    return res.send('<h2>로그인이 필요합니다.</h2>');  
  }  
}
```

```
const { id } = req.params;  
const { title, content } = req.body;
```

```
try {  
  const [rows] = await pool.query('SELECT * FROM articles WHERE id = ?', [id]); // 작성자 확인  
  if (rows.length === 0) return res.send('<h2>게시글이 존재하지 않습니다.</h2>');  
  if (rows[0].writer !== req.session.user.username) {  
    return res.send('<h2>본인이 작성한 글만 수정할 수 있습니다.</h2>');  
  }
```

```
  const sql = 'UPDATE articles SET title = ?, content = ? WHERE id = ?';  
  await pool.query(sql, [title, content, id]);
```

```
  res.redirect(`/board/${id}`);  
} catch (err) {  
  console.error('수정 처리 오류:', err.message);  
  res.send('<h2>글 수정 중 오류가 발생했습니다.</h2>');  
}  
});
```

게시글 삭제

□ 기능 개요

- 경로: **GET** `/board/:id/delete`
- 로그인한 사용자만 가능
- 작성자만 삭제 가능

□ routes/board.js

// 게시글 삭제

```
router.get('/:id/delete', async (req, res) => {
  if (!req.session.user) {
    return res.send('<h2>로그인 후 삭제할 수 있습니다. <a href="/signin">로그인</a></h2>');
  }

  const { id } = req.params;

  try {
    const [rows] = await pool.query('SELECT * FROM articles WHERE id = ?', [id]);

    if (rows.length === 0) {
      return res.send('<h2>존재하지 않는 게시글입니다.</h2>');
    }

    const article = rows[0];

    if (article.writer !== req.session.user.username) {
      return res.send('<h2>본인이 작성한 글만 삭제할 수 있습니다.</h2>');
    }

    await pool.query('DELETE FROM articles WHERE id = ?', [id]);
    res.redirect('/board');
  } catch (err) {
    console.error('삭제 오류:', err.message);
    res.send('<h2>글 삭제 중 오류가 발생했습니다.</h2>');
  }
});
```

□ views/board-detail.ejs

<h2><%= article.title %></h2>

<p style="color: gray;">

작성자: <%= article.writer %> |

작성일: <%= new Date(new Date(article.created_at).getTime() + 9 * 60 * 60 * 1000).toLocaleString('ko-KR') %>

</p>

<div style="margin-top: 20px; white-space: pre-wrap;">

<%= article.content %>

</div>

<% if (user && user.username === article.writer) { %>

<p style="text-align: right;">

<a href="/board/<%= article.id %>/edit">✎ 수정

|

<a href="/board/<%= article.id %>/delete" onclick="return confirm('정말 삭제하시겠습니까?');">

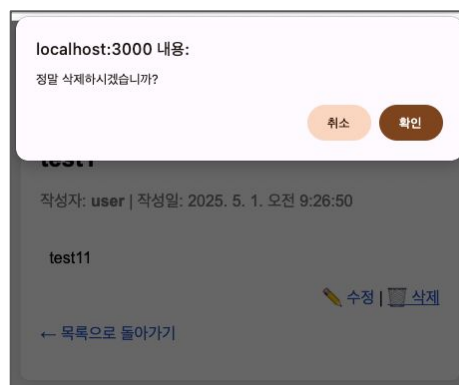


삭제

</p>

<% } %>

<p class="message">← 목록으로 돌아가기</p>



게시글 조회수 증가

- 기능 개요
 - 하루에 한 번만 조회수 증가 (로그인 사용자 기준)

- 테이블 구성
 - articles 테이블에 조회수 정보 컬럼 추가
`ALTER TABLE articles ADD views INT DEFAULT 0;`
 - views_log 테이블에 조회자 정보 저장
`CREATE TABLE IF NOT EXISTS views_log (
 id INT AUTO_INCREMENT PRIMARY KEY,
 user_id INT NOT NULL,
 article_id INT NOT NULL,
 view_date DATE NOT NULL,
 UNIQUE KEY unique_view (user_id, article_id, view_date)
);`

항목	설명
articles.views	빠른 화면 표시용 캐시
views_log	사용자별 중복 조회 방지 + 정확한 기록
트랜잭션 사용	동기화 오류 방지
기준	로그인 사용자당 하루 1회만 조회수 증가
비로그인 사용자	조회수 증가 대상 아님 (정확한 기준이 없기 때문)

□ routes/board.js

// 게시물 상세 보기

```
router.get('/:id', async (req, res) => {
  const { id } = req.params;
  const user = req.session.user;

  try {
    const sql = 'SELECT * FROM articles WHERE id = ?';
    const [rows] = await pool.query(sql, [id]);

    if (rows.length === 0) {
      return res.send('<h2>존재하지 않는 게시물입니다. <a href="/board">목록으로</a></h2>');
    }

    const article = rows[0];

    // 조회수 증가 처리
    if (user) {
      const conn = await pool.getConnection();
      try {
        await conn.beginTransaction();

        const sqlCheck = `
          SELECT * FROM views_log
          WHERE user_id = ? AND article_id = ? AND view_date = CURDATE()
        `;
        const [logs] = await conn.query(sqlCheck, [user.id, id]);

        if (logs.length === 0) {
          await conn.query('UPDATE articles SET views = views + 1 WHERE id = ?', [id]);
          await conn.query(
            'INSERT INTO views_log (user_id, article_id, view_date) VALUES (?, ?, CURDATE())',
            [user.id, id]
          );
        }

        await conn.commit();
      } catch (err) {
        await conn.rollback();
        console.error('조회수 트랜잭션 오류:', err.message);
      } finally {
        conn.release();
      }
    }

    res.render('board-detail', { article });
  } catch (err) {
    console.error('상세보기 오류:', err.message);
    res.send('<h2>게시물을 불러오는 데 실패했습니다.</h2>');
  }
});
```

□ views/board-detail.ejs (상세 화면 조회수 표시)

<p style="color: gray;">

작성자: <%= article.writer %> |

작성일: <%= new Date(new Date(article.created_at).getTime() + 9 * 60 * 60 * 1000).toLocaleString('ko-KR') %>

조회수: <%= article.views %>

</p>

□ views/board-list.ejs (목록 화면 조회수 표시)

<table border="1" cellspacing="0" cellpadding="8" width="100%">

<tr>

<th>번호</th>

<th>제목</th>

<th>작성자</th>

<th>작성일</th>

<th>조회수</th>

</tr>

<% articles.forEach(article => { %>

<tr>

<td><%= article.id %></td>

<td><a href="/board/<%= article.id %>"><%= article.title %></td>

<td><%= article.writer %></td>

<td>

<%=

new Date(

new Date(article.created_at).getTime() + 9 * 60 * 60 * 1000

).toLocaleString('ko-KR')

%>

</td>

<td><%= article.views %></td> <!-- 조회수 추가 -->

</tr>

<% }) %>

</table>

<% } %>



□ routes/board.js (목록 화면 조회수 데이터 추가)

// 게시글 목록 보기

router.get('/', async (req, res) => {

try {

const sql = 'SELECT id, title, writer, created_at, views FROM articles ORDER BY id DESC';

const [articles] = await pool.query(sql);

res.render('board-list', { articles });

} catch (err) {

console.error('게시글 목록 조회 오류:', err.message);

res.send('<h2>게시글을 불러오는 데 실패했습니다.</h2>');

}

});

게시글 좋아요 / 싫어요 기능

□ 기능 개요

- 경로: `/board/:id/like`, `/board/:id/dislike`
- 로그인 사용자만 가능
- 하루 한 번이 아닌, **한 사용자당 한 번만** 가능
- 좋아요/싫어요 상태는 `likes_log` 테이블에 저장
- `articles` 테이블에는 좋아요/싫어요 수를 별도로 기록

□ 테이블 구성

- `articles` 테이블에 좋아요, 싫어요 정보 컬럼 추가

```
ALTER TABLE articles
ADD likes INT DEFAULT 0,
ADD dislikes INT DEFAULT 0;
```

- `likes_log` 테이블에 평가자 정보 저장

```
CREATE TABLE IF NOT EXISTS likes_log (
  id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL,
  article_id INT NOT NULL,
  type ENUM('like', 'dislike') NOT NULL,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY unique_like (user_id, article_id)
);
```

항목	설명
좋아요/싫어요 수	<code>articles.likes</code> , <code>articles.dislikes</code>
중복 방지	<code>likes_log</code> 의 <code>UNIQUE(user_id, article_id)</code> 제약
로그인 필수	비회원은 평가 불가
하루 1회 제한 아님	한 번만 가능하게 설정됨 (변경도 차후 가능)

□ routes/board.js

// 좋아요 처리

```
router.get('/:id/like', async (req, res) => {  
  if (!req.session.user) {  
    return res.send('<h2>로그인 후 좋아요를 누를 수 있습니다. <a href="/signin">로그인</a></h2>');  
  }  
  
  const { id } = req.params;  
  const userId = req.session.user.id;  
  
  try {  
    const [existing] = await pool.query(  
      'SELECT * FROM likes_log WHERE user_id = ? AND article_id = ?',  
      [userId, id]  
    );  
  
    if (existing.length > 0) {  
      return res.send('<h2>이미 평가한 게시글입니다. <a href="/board/' + id + '">돌아가기</a></h2>');  
    }  
  
    await pool.query('UPDATE articles SET likes = likes + 1 WHERE id = ?', [id]);  
    await pool.query('INSERT INTO likes_log (user_id, article_id, type) VALUES (?, ?, ?)', [userId, id,  
      'like']);  
    res.redirect('/board/' + id);  
  } catch (err) {  
    console.error('좋아요 처리 오류:', err.message);  
    res.send('<h2>좋아요 처리 중 오류가 발생했습니다.</h2>');  
  }  
});
```

□ routes/board.js

// 싫어요 처리

```
router.get('/:id/dislike', async (req, res) => {  
  if (!req.session.user) {  
    return res.send('<h2>로그인 후 싫어요를 누를 수 있습니다. <a href="/signin">로그인</a></h2>');  
  }  
  
  const { id } = req.params;  
  const userId = req.session.user.id;  
  
  try {  
    const [existing] = await pool.query(  
      'SELECT * FROM likes_log WHERE user_id = ? AND article_id = ?',  
      [userId, id]  
    );  
  
    if (existing.length > 0) {  
      return res.send('<h2>이미 평가한 게시글입니다. <a href="/board/' + id + '">돌아가기</a></h2>');  
    }  
  
    await pool.query('UPDATE articles SET dislikes = dislikes + 1 WHERE id = ?', [id]);  
    await pool.query('INSERT INTO likes_log (user_id, article_id, type) VALUES (?, ?, ?)', [userId, id,  
      'dislike']);  
    res.redirect('/board/' + id);  
  } catch (err) {  
    console.error('싫어요 처리 오류:', err.message);  
    res.send('<h2>싫어요 처리 중 오류가 발생했습니다.</h2>');  
  }  
});
```



댓글

□ 기능 개요

- 경로: `/board/:id/comment`
- 로그인 사용자만 가능

□ 테이블 구성

- `articles` 테이블에 댓글 개수 정보 컬럼 추가
`ALTER TABLE articles ADD comment_count INT DEFAULT 0;`

- `comments` 테이블에 댓글 정보 저장
`CREATE TABLE IF NOT EXISTS comments (
 id INT AUTO_INCREMENT PRIMARY KEY,
 article_id INT NOT NULL,
 user_id INT NOT NULL,
 content TEXT NOT NULL,
 created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
 FOREIGN KEY (article_id) REFERENCES articles(id) ON DELETE CASCADE,
 FOREIGN KEY (user_id) REFERENCES members(id) ON DELETE CASCADE
);`

항목	내용
댓글 저장 테이블	<code>comments (article_id, user_id, content, created_at)</code>
게시글과 연결	<code>FOREIGN KEY(article_id) → articles(id)</code>
사용자와 연결	<code>FOREIGN KEY(user_id) → members(id)</code>
댓글 수 표시용 컬럼	<code>articles.comment_count</code>

□ routes/board.js

// 댓글 등록

```
router.post('/:id/comment', async (req, res) => {
  if (!req.session.user) {
    return res.send('<h2>로그인 후 댓글을 작성할 수 있습니다. <a href="/signin">로그인</a></h2>');
  }

  const { id } = req.params;
  const { content } = req.body;
  const userId = req.session.user.id;

  const conn = await pool.getConnection();
  try {
    await conn.beginTransaction();

    await conn.query(
      'INSERT INTO comments (article_id, user_id, content) VALUES (?, ?, ?)',
      [id, userId, content]
    );

    await conn.query(
      'UPDATE articles SET comment_count = comment_count + 1 WHERE id = ?',
      [id]
    );

    await conn.commit();
    res.redirect('/board/' + id);
  } catch (err) {
    await conn.rollback();
    console.error('댓글 등록 오류:', err.message);
    res.send('<h2>댓글 등록 중 오류가 발생했습니다.</h2>');
  } finally {
    conn.release();
  }
});
```

□ routes/board.js

// 게시물 상세 보기

```
router.get('/:id', async (req, res) => {
  const { id } = req.params;
  const user = req.session.user;

  try {
    const sql = 'SELECT * FROM articles WHERE id = ?';
    const [rows] = await pool.query(sql, [id]);

    if (rows.length === 0) {
      return res.send('<h2>존재하지 않는 게시물입니다. <a href="/board">목록으로</a></h2>');
    }

    const article = rows[0];

    ...

    // 댓글 목록 조회
    const [comments] = await pool.query(
      `SELECT c.*, m.username
      FROM comments c
      JOIN members m ON c.user_id = m.id
      WHERE c.article_id = ?
      ORDER BY c.id ASC`,
      [id]
    );

    res.render('board-detail', { article, comments });
  } catch (err) {
    console.error('상세보기 오류:', err.message);
    res.send('<h2>게시글을 불러오는 데 실패했습니다.</h2>');
  }
});
```

□ views/board-detail.ejs

...

<p class="message">← 목록으로 돌아가기</p>

<h3>💬 댓글 (<%= article.comment_count %>)</h3>

<% if (comments.length === 0) { %>

<p class="message">등록된 댓글이 없습니다.</p>

<% } else { %>

<% comments.forEach(c => { %>

<li style="margin-bottom: 8px;">

<%= c.username %>:

<%= c.content %>

<small style="color: gray; margin-left: 10px;">

<%= new Date(new Date(c.created_at).getTime() + 9 * 60 * 60 * 1000).toLocaleString("ko-KR") %>

</small>

<% }) %>

<% } %>

<% if (user) { %>

<form method="POST" action="/board/<%= article.id %>/comment" style="margin-top: 20px;">

<textarea name="content" rows="3" required placeholder="댓글을 입력하세요"></textarea>

<button type="submit">댓글 작성</button>

</form>

<% } else { %>

<p class="message">댓글을 작성하려면 로그인 해주세요.</p>

<% } %>

user님, 안녕하세요! 😊 로그아웃

test1

작성자: user | 작성일: 2025. 5. 1. 오전 9:26:50 조회수: 1

test11

👍 좋아요: 1 🖱️ 싫어요: 0

👍 좋아요 | 🖱️ 싫어요

✏️ 수정 | 🗑️ 삭제

← 목록으로 돌아가기

💬 댓글 (0)

등록된 댓글이 없습니다.

댓글을 입력하세요

댓글 작성

test1

작성자: user | 작성일: 2025. 5. 1. 오전 9:26:50 조회수: 1

test11

👍 좋아요: 1 🖱️ 싫어요: 0

← 목록으로 돌아가기

💬 댓글 (0)

등록된 댓글이 없습니다.

댓글을 작성하려면 로그인 해주세요.